

Marcos Roberto Braga
Com a colaboração de Pedro Henrique Cerqueira Braga (Testes)

Edição 2024 — Atualizada em 2026

2026

- ➊ **A tese central** — tempo real é físico, não abstrato
- ➋ **Os 8 fundamentos** — tarefas atômicas, despachador, semáforos, etc.
- ➌ **As 7 camadas de proteção** — auto-regulagem, Síndico, Xerife, etc.
- ➍ **Casos de uso** — uOS, Robótica, TCC.C, CDC-P
- ➎ **Comparação com RTOS preemptivos**
- ➏ **Quando usar** — e quando não usar

“Tempo real não é uma abstração que o software mantém.
É um fenômeno físico que o hardware gera.
O software apenas coordena.”

- **Modelo abstrato:** RTOS preemptivos, escalonadores dinâmicos, filas, mutexes
- **Modelo físico:** tarefas atômicas, ordem fixa, semáforos por bit, bases temporais separadas

O problema dos RTOS preemptivos

- **Jitter:** o tempo varia (depende da carga, do escalonador, das interrupções)
- **Deriva:** o tempo acumula erro (o tick é uma contagem, não uma medida física)
- **Dependência:** o tempo depende da saúde do escalonador
- **Deadlock:** possível (há mutexes e espera)
- **Inversão de prioridade:** possível (há recursos compartilhados)
- **Pior caso:** complexo de calcular (depende do estado do sistema)

Em sistemas críticos, essas propriedades são fatais.

Definição

Cada tarefa executa até o fim e sempre retorna. Nunca é interrompida no meio.

O que isso elimina:

- Contexto salvo
- Stack por tarefa
- TCB (Task Control Block)
- Mutex
- Deadlock
- Inversão de prioridade

Definição

Laço infinito que verifica cada tarefa em ordem fixa, chamando-a quando for a hora.

Estrutura:

- **Condição:** verifica se é hora (e se a tarefa não está bloqueada)
- **Atualização:** atualiza o tick da última execução
- **Chamada:** executa a tarefa

Quatro níveis de prioridade:

- 1 URG-S (urgência situacional)
- 2 Preempção cooperativa
- 3 Kernel (Síndico e Xerife)
- 4 Aplicação (ordem espacial fixa)

Definição

Semáforo implementado como bit em um registrador global.

Três operações:

- **SETAR** (bloquear): `semaforos |= SEMA_TASKN_MASK;`
- **LIMPAR** (desbloquear): `semaforos &= ~SEMA_TASKN_MASK;`
- **TESTAR**: `if (semaforos & SEMA_TASKN_MASK) { ... }`

Vantagens:

- Sem deadlock (não há espera)
- Sem inversão de prioridade (não há mutex)
- Uma instrução de hardware

Definição

Duas bases distintas: o **tick** (para tarefas de curto período) e o **segundo** (para tarefas de longo período).

Base	Período típico	Uso
Tick	2 a 8 ms	Tarefas curtas
Segundo	1 a 60 s	Tarefas longas

Vantagens:

- Contadores pequenos (`uint8_t` ou `uint16_t`)
- Precisão preservada em cada escala

Definição

A prioridade tem três dimensões: espacial, temporal e situacional.

Espacial: posição fixa no despachador (ordem de verificação)

Temporal: período configurado (frequência de execução)

Situacional: flag de urgência (URG-S)

Combinação:

- Urgência sobrepõe período
- Período define frequência
- Ordem define desempate

Definição

Propriedade do hardware (e formalizada em C) que faz contadores voltarem a zero após um limite.

Aplicações no CDC-P:

- Comparação de tempo (`tick - timer_taskX`)
- Máscaras de bit (`semaforos & MASK`)
- Contadores pequenos (`uint8_t`)

Exemplo (wrap-around em 8 bits):

- `tick = 3, timer_task1 = 250`
- `3 - 250 = 9` (módulo 256) — **correto!**

Definição

Flag que indica que uma tarefa deve executar imediatamente, independentemente de seu período.

Origem:

- ISRs (interrupções externas)
- Tarefas (uma tarefa pode setar urgência de outra)
- Comandos externos (via serial)

Latência: um ciclo do despachador (conhecida e limitada).

Definição

Acelera uma tarefa sem interrompê-la, alterando seu período temporariamente.

Como funciona:

- Salvar o período original
- Reduzir o período para 1 tick
- A tarefa passa a executar com frequência máxima
- Ao desativar, restaurar o período original

Vantagens:

- Zero contexto
- Zero condições de corrida
- Zero deadlock

As 7 camadas de proteção

- 1 Auto-regulagem temporal — cada tarefa se desacelera se atrasa
- 2 Bloqueio por violação de deadline — tarefas que excedem o tempo máximo
- 3 Diagnóstico nativo (Xerife) — tarefas cronicamente problemáticas
- 4 Recuperação progressiva (Síndico) — tarefas bloqueadas
- 5 Fail-safe final (reset_cpu) — se o Síndico falha
- 6 Aceleração automática (preempção) — resposta a comandos
- 7 URG-S (urgência situacional) — eventos urgentes

Cada camada captura um problema diferente.

Caso	MCU	RAM	Flash	Tarefas
uOS (2011)	AT90S8515	512 B	8 KB	20
Robótica (2012)	PIC16F84	68 B	1 KB	6
TCC.C (2013)	PIC18F6722	3,9 KB	128 KB	3+
CDC-P (2024+)	CH32V003	2 KB	16 KB	6+2

De 2011 a 2024: de um experimento a um sistema consolidado.

Comparação com RTOS preemptivos

Critério	RTOS Preemptivo	CDC-P
Modelo de tarefa	TCB + stack	Função void
RAM por tarefa	100–500 bytes	~5 bytes
Flash do kernel	10–20 KB	2 KB
Sincronização	Mutex, fila	Semáforo por bit
Deadlock	Possível	Impossível
Escalonamento	Dinâmico	Ordem fixa
Pior caso	Complexo (RTA)	Trivial

Quando usar — e quando não usar

Usar o CDC-P quando:

- Conjunto fixo de tarefas
- Recursos escassos
- Determinismo estrito
- Sistema crítico de longa duração
- Alto volume

Não usar quando:

- Muitas tarefas dinâmicas
- Prioridades dinâmicas complexas
- Comunicação complexa
- Soft real-time
- Reuso de código (TCP/IP, USB)

“O CDC-P não inventou nada. Organizou o que já existia.”

O CDC-P é:

- Uma síntese de qualidades conhecidas
- Uma alternativa viável onde RTOS preemptivos não cabem
- Um método testado em produtos reais
- Uma ferramenta para contextos específicos

- Baker, T. P., and Shaw, A. *The Cyclic Executive Model and Ada*. Real-Time Systems, 1989.
- Liu, C. L., and Layland, J. W. *Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment*. Journal of the ACM, 1973.
- Kopetz, H., and Bauer, G. *The Time-Triggered Architecture*. Proceedings of the IEEE, 2003.
- Kephart, J. O., and Chess, D. M. *The Vision of Autonomic Computing*. IEEE Computer, 2003.
- ARINC Specification 653: *Avionics Application Software Standard Interface*. 2015.

Obrigado!

Marcos Roberto Braga

GitHub: <https://github.com/Noelmrbs-tec-dotcom/cdc-p-source>

E-mail: noelmrbs_tec@yahoo.com